

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit I | Session 2 | SLO 2: Abstraction & Specification (Code: 1022)

Q: 1. Conceptual: Why is strong typing considered a significant design choice in modern programming languages?

Ans: Explanation: Strong typing ensures that the language strictly forbids implicit, unsafe type conversions (coercions) that could corrupt data or lead to unexpected behavior. It enhances code correctness, memory safety, predictability, and prevents subtle runtime bugs by forcing explicit conversions.

Q: 2. Analytical: Compare static typing and dynamic typing. How do these affect error detection and program reliability?

Ans: Comparison:

- **Static Typing (e.g., Java, C++):** Types are bound and verified at compile time. Syntax and type mismatch bugs are caught early before execution. IDEs provide accurate autocompletion and refactoring, yielding higher software reliability for enterprise systems.
- **Dynamic Typing (e.g., Python, JavaScript):** Types are associated with runtime values rather than variables. Offers rapid prototyping and concise syntax, but type errors surface only during runtime execution, requiring extensive test suite coverage.

Q: 3. Critical Thinking: Java and Python use different syntax styles. How does syntax design impact programmer productivity and code maintainability?

Ans: Analysis:

- **Python's Syntax:** Relies on meaningful indentation and omission of semicolons/parentheses. This minimizes visual boilerplate, speeding up initial development and prototyping.
- **Java's Syntax:** Utilizes explicit C-family braces ({}) and mandatory type declarations. While more verbose, it provides clear, unambiguous structural boundaries that facilitate maintainability and large-scale multi-developer collaboration.